

Java — Advanced Fundamentals

51. What is an abstract class?

Answer:

An abstract class is a class declared with abstract that cannot be instantiated directly. It can contain both abstract methods without a body and concrete methods with implementation.

52. What is an interface?

Answer:

An interface defines a contract that implementing classes must follow. A class can implement multiple interfaces, which allows Java to achieve multiple inheritance of type.

53. Interface vs abstract class?

Answer:

Abstract Class	Interface
Can have abstract + concrete methods	Primarily defines a contract; can also have default/static methods
Can have instance variables	Fields are implicitly public static final
Extended using extends	Implemented using implements
A class extends only one class	A class can implement multiple interfaces

54. Can we create an object of an abstract class?

Answer:

No. An abstract class cannot be instantiated directly.

```
Animal a = new Animal(); // invalid if Animal is abstract
```

But we can create a reference of the abstract class pointing to a child object:

```
Animal a = new Dog();
```

55. Can an abstract class have a constructor?

Answer:

Yes. Its constructor is executed when a child-class object is created, usually as part of the superclass initialization.

56. Can an interface have a constructor?

Answer:

No. An interface cannot be instantiated, so it does not have constructors.

57. Can we override a static method?

Answer:

No. Static methods belong to the class, not the object. A child class can define a static method with the same signature, but that is **method hiding**, not overriding.

58. Can we override a private method?

Answer:

No. Private methods are not accessible to child classes, so they cannot be overridden.

59. Can a final method be overridden?

Answer:

No. A final method cannot be overridden by a child class.

60. Can a final class be inherited?

Answer:

No.

```
final class A {  
}
```

Another class cannot extend A.

A common example is String.

Java Collections

61. Array vs ArrayList?

Answer:

An array has a fixed size, while ArrayList is dynamically resizable.

Array → fixed size
ArrayList → dynamic size

Array can store primitives directly, while ArrayList stores objects/wrapper types.

62. HashMap vs Hashtable?

Answer:

- HashMap is not synchronized by default.
- Hashtable is synchronized.
- HashMap allows one null key and multiple null values.
- Hashtable does not allow null keys or null values.
- HashMap is generally preferred in modern Java when synchronization is not specifically required.

63. What is LinkedHashMap?

Answer:

LinkedHashMap is a HashMap implementation that maintains a predictable iteration order, normally insertion order.

64. What is TreeMap?

Answer:

TreeMap stores key-value pairs sorted according to the keys' natural ordering or a supplied comparator.

Typical lookup/insertion/deletion complexity is **O(log n)**.

65. HashMap vs TreeMap?

Answer:

HashMap	TreeMap
No sorted ordering	Sorted by key
Average O(1) lookup	O(log n) lookup
Hash table based	Tree based
Allows one null key	Natural ordering generally does not permit null keys

66. What is Comparable vs Comparator?

Answer:

Comparable defines the object's **natural ordering** inside the class.

class Student implements Comparable<Student>

Comparator defines an **external/custom ordering**.

Comparator<Student>

A class can have one natural ordering but many different comparators.

67. What is an Iterator?

Answer:

An Iterator is used to traverse elements of a collection sequentially.

```
Iterator<Integer> it = list.iterator();
```

```
while(it.hasNext()) {  
    System.out.println(it.next());  
}
```

68. What is ConcurrentModificationException?

Answer:

It can occur when a collection is structurally modified while it is being iterated in an unsupported way.

For example, directly removing an element from an ArrayList during a for-each loop can cause it.

Using Iterator.remove() is one appropriate way to remove during iteration.

69. What is fail-fast behavior?

Answer:

A fail-fast iterator detects certain structural modifications to a collection during iteration and may throw ConcurrentModificationException.

It is a detection mechanism, not a synchronization guarantee.

70. How does HashMap work internally?

Answer:

At a high level:

```
key  
↓  
hashCode()  
↓  
hash calculation  
↓  
bucket  
↓  
compare keys using equals()  
↓  
store/find value
```

In modern Java, heavily-collided buckets can be converted from linked structures into balanced tree structures under certain conditions.

Spring Boot

71. What is @SpringBootApplication?

Answer:

@SpringBootApplication is a convenience annotation that combines three major annotations:

@Configuration
@EnableAutoConfiguration
@ComponentScan

It is commonly placed on the main Spring Boot application class.

72. What is auto-configuration?

Answer:

Spring Boot automatically configures application components based on the dependencies present in the classpath and the application's configuration.

For example, adding a web starter allows Spring Boot to configure much of the infrastructure required for a web application.

73. What are Spring Boot starter dependencies?

Answer:

Starters are dependency bundles that simplify dependency management.

For example:

spring-boot-starter-web

provides dependencies commonly needed to build a Spring web application.

74. What is application.properties / application.yml used for?

Answer:

They are commonly used to configure application settings such as:

- Database connection
 - Server port
 - Logging
 - Spring settings
 - Custom application properties
-

75. What is REST API?

Answer:

A REST API is an API designed around resources and HTTP operations.

Common HTTP methods:

GET → retrieve
POST → create
PUT → replace/update
PATCH → partial update
DELETE → delete

76. What is @RequestBody?

Answer:

@RequestBody tells Spring to deserialize the HTTP request body into a Java object.

Example:

```
@PostMapping("/students")
public Student create(@RequestBody Student student) {
    ...
}
```

77. What is @PathVariable?

Answer:

It extracts a value from the URL path.

```
/students/25
@GetMapping("/students/{id}")
public Student getStudent(@PathVariable Long id) {
    ...
}
```

Here id = 25.

78. @RequestParam vs @PathVariable?

Answer:

```
/students/25
```

uses a **PathVariable**.

```
/students?id=25
```

uses a **RequestParam**.

So:

@PathVariable
@RequestParam

represent two different ways of passing request data.

79. What is Spring Data JPA?

Answer:

Spring Data JPA simplifies database access using JPA repositories.

Instead of writing a large amount of boilerplate DAO code, we can define repository interfaces and use methods such as:

```
findByEmail(...)
findById(...)
save(...)
delete(...)
```

80. What is JPA vs Hibernate?

Answer:

JPA is a Java persistence specification/API.

Hibernate is an implementation of JPA.

So:

JPA → specification
Hibernate → implementation

81. What is @Entity?

Answer:

@Entity marks a Java class as a JPA entity that is mapped to a database table.

```
@Entity
class Student {
}
```

82. What is @Id?

Answer:

@Id identifies the field that acts as the primary key of a JPA entity.

```
@Id
private Long studentId;
```

83. What is DTO and why use it?

Answer:

DTO means **Data Transfer Object**.

It is used to transfer only the required data between layers or between the client and server.

Using DTOs can:

- Prevent exposing internal entity structure
- Control request/response fields

- Improve API design
 - Help avoid accidentally exposing sensitive fields
-

84. Entity vs DTO?

Answer:

Entity represents persistent database data.

DTO represents data transferred between application boundaries.

For example, a User entity might contain:

```
id
email
passwordHash
role
```

But the API response DTO should not expose the password hash.

85. What is @ExceptionHandler?

Answer:

@ExceptionHandler is used to handle specific exceptions in Spring MVC controllers.

For centralized exception handling, @ControllerAdvice or @RestControllerAdvice can be used.

SQL / DBMS

86. What is a transaction?

Answer:

A transaction is a logical unit of database work that should be completed according to the database's transaction rules.

Example:

```
Debit account A
Credit account B
```

Both operations should succeed together or the transaction should be rolled back.

87. What is a candidate key?

Answer:

A candidate key is a minimal set of attributes that can uniquely identify a row.

One candidate key is selected as the primary key.

88. What is a composite key?

Answer:

A composite key consists of multiple columns that together uniquely identify a row.

Example:

student_id + project_id

89. What is a unique constraint?

Answer:

A unique constraint ensures that duplicate values are not allowed in the constrained column or column combination.

Example:

email VARCHAR(255) UNIQUE

90. WHERE vs HAVING?

Answer:

WHERE filters individual rows **before grouping**.

HAVING filters groups **after aggregation**.

WHERE salary > 50000

filters rows.

HAVING COUNT(*) > 5

filters groups.

91. UNION vs UNION ALL?

Answer:

UNION combines results and removes duplicates.

UNION ALL combines results without removing duplicates and is generally faster because duplicate elimination isn't performed.

92. What is a subquery?

Answer:

A subquery is a query nested inside another SQL query.

Example:

```
SELECT name
FROM Employee
WHERE salary > (
    SELECT AVG(salary)
    FROM Employee
);
```

93. What is a view?

Answer:

A view is a virtual table based on a SQL query.

It can simplify complex queries and control what data users can access.

94. What is a stored procedure?

Answer:

A stored procedure is a stored program consisting of SQL statements that can be executed by the database.

It can encapsulate reusable database-side operations.

95. What are database anomalies?

Answer:

Three common anomalies caused by poor database design are:

1. **Insert anomaly**
2. **Update anomaly**
3. **Delete anomaly**

Normalization helps reduce these problems.

Computer Networks

96. What are the OSI layers?

Answer:

There are seven OSI layers:

7. Application
6. Presentation
5. Session
4. Transport
3. Network
2. Data Link
1. Physical

Memory:

All People Seem To Need Data Processing

97. OSI vs TCP/IP model?

Answer:

OSI is a **7-layer reference model**.

TCP/IP is the practical protocol suite/model used for Internet networking and is commonly represented using fewer layers.

A common mapping is:

OSI Application
OSI Presentation → TCP/IP Application
OSI Session

OSI Transport → Transport
OSI Network → Internet
OSI Data Link
OSI Physical → Link/Network Access

98. What is a MAC address?

Answer:

A MAC address is a hardware/network-interface-level address used for communication on a local network.

It operates at the **Data Link layer**.

IP addresses are used for logical network addressing and routing.

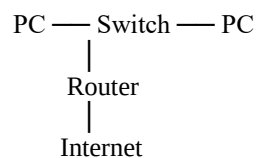
99. What is the difference between a switch and a router?

Answer:

A **switch** primarily connects devices within a local network and forwards frames using MAC addresses.

A **router** connects different networks and forwards packets using IP addresses.

Simple example:



100. What happens when you enter a URL in a browser?

Answer:

A strong interview-level answer:

Enter URL
↓
DNS lookup
↓
Obtain server IP
↓
Establish connection
↓
TLS handshake if HTTPS
↓
Send HTTP request
↓
Server processes request
↓
HTTP response
↓
Browser renders response

For HTTPS, TLS provides encryption and server authentication.
